

S140-B - Cross-family predicate invention

Wolfram source and recorded execution result

Original three-family diagnostic. The recorded aggregate remains the authoritative failure/pass record.

Wolfram runner source

```
ClearAll["Global`*"];

rootDirectory = DirectoryName[DirectoryName[$InputFileName]];
protocolPath = FileNameJoin[{rootDirectory, "protocol", "frozen_protocol.json"}];
publicPath = FileNameJoin[{rootDirectory, "input", "public_tasks.json"}];
resultPath = FileNameJoin[{rootDirectory, "results", "kernel_native_concept_result.json"}];
conceptLibraryPath = FileNameJoin[{rootDirectory, "library", "kernel_induced_concepts.json"}];
oracleResponderPath = FileNameJoin[{rootDirectory, "source", "oracle_responder.py"}];
oracleRequestPath = FileNameJoin[{rootDirectory, "oracle", "runtime_request.json"}];
oracleResponsePath = FileNameJoin[{rootDirectory, "oracle", "runtime_response.json"}];
oracleLogPath = FileNameJoin[{rootDirectory, "oracle", "query_log.jsonl"}];
protocol = Import[protocolPath, "RawJSON"];
public = Import[publicPath, "RawJSON"];
protocolHash = IntegerString[FileHash[protocolPath, "SHA256"], 16, 64];
runnerHash = IntegerString[FileHash[$InputFileName, "SHA256"], 16, 64];
oracleHash = IntegerString[FileHash[oracleResponderPath, "SHA256"], 16, 64];
If[public["ProtocolSHA256"] != protocolHash, Print["FATAL: public/protocol mismatch"]; Exit[2]];
If[runnerHash != protocol["FrozenWolframRunnerSHA256"], Print["FATAL: runner differs from frozen protocol"]; Exit[3]];
If[oracleHash != protocol["FrozenOracleResponderSHA256"], Print["FATAL: oracle differs from frozen protocol"]; Exit[4]];

ContextColorsS139[level_Integer] := {6, 7, 9}[[Mod[level, 3] + 1]];

GridDigestG3[grid_List] := IntegerString[Hash[StringRiffle[ToString /@ Flatten[grid], ",", "SHA256"], 16, 64];
AllPrefixesG3[counts_List, 0] := {};
AllPrefixesG3[counts_List, length_Integer] := Tuples[Table[Range[0, counts[[index]] - 1], {index, length}]];

MakeInputG3[task_Association, spec_Association] := Module[
  {h = task["GridHeight"], w = task["GridWidth"], grid, shape, top, left, kind, prefix},
  grid = ConstantArray[0, {h, w}];
  Do[Do[grid[[h - level, 3 + 4 level + value]] = ContextColorsS139[level],
    {value, 0, task["ContextCounts"][[level + 1]] - 1}], {level, 0, Length[task["ContextCounts"]] - 1}];
  kind = spec["Kind"];
  shape = Which[kind == "TEST", task["TestShape"], kind == "TRAIN", task["TrainShape"], True, task["ProbeShape"]];
  top = task["TargetTop"]; left = task["TargetLeft"];
  Do[grid[[top + cell[[1]] + 1, left + cell[[2]] + 1]] = task["TargetColor"], {cell, shape}];
  prefix = Lookup[spec, "Prefix", {}];
  Do[grid[[2 + level, 3 + 4 level + prefix[[level + 1]]]] = 8, {level, 0, Length[prefix] - 1}];
  Which[
    kind == "CALIBRATE", grid[[1, spec["Level"] + 1]] = 9,
    kind == "DECISION", grid[[1, w]] = 8,
    kind == "GLOBAL_DECISION", grid[[1, w - 1]] = 4,
    kind == "GLOBAL_BIT0", grid[[1, w - 1]] = 8,
    kind == "GLOBAL_BIT1", grid[[1, w - 2]] = 8,
    kind == "NUISANCE", grid[[1, w]] = 7,
    kind == "TEST", grid[[1, w]] = 5,
    kind == "TRAIN", Null,
    True, Print["FATAL: unknown spec kind"]; Exit[10]
  ]; grid
];

ApplyProgramG3[task_Association, program_Association, spec_Association] := Module[
  {grid, output, kind, prefix, level, value, transform, positions, r0, r1, c0, c1, mapped},
  grid = MakeInputG3[task, spec]; output = grid; kind = spec["Kind"];
  prefix = Lookup[spec, "Prefix", {}];
  If[kind == "CALIBRATE", level = spec["Level"];
    If[!MemberQ[Lookup[task, "CorruptedCalibrationLevels", {}], level] &&
      Take[program["Keys"], level] == prefix, value = program["Keys"][[level + 1]];
      output[[6 + level, 3 + 4 level + value]] = ContextColorsS139[level]; Return[output]];
  If[kind == "NUISANCE", output[[9, 13 + program["Nuisance"]]] = 9; Return[output]];
  If[MemberQ[{"GLOBAL_BIT0", "GLOBAL_BIT1"}, kind],
    If[program["Decision"] == If[kind == "GLOBAL_BIT0", 0, 1],
      output[[10, Length[First[output]] - 1]] = 9; Return[output]];
  transform = MemberQ[{"TRAIN", "TEST", "GLOBAL_DECISION"}, kind] ||
    (kind == "DECISION" && Take[program["Keys"], Length[prefix]] == prefix);
  If[!transform, Return[output]];
  positions = Position[grid, task["TargetColor"], {2}];
  r0 = Min[positions[[All, 1]]]; r1 = Max[positions[[All, 1]]];
  c0 = Min[positions[[All, 2]]]; c1 = Max[positions[[All, 2]]];
```

```

mapped = Switch[program["Decision"], 0, positions,
  1, {#[[1]], c0 + c1 - #[[2]]} & /@ positions,
  2, {r0 + r1 - #[[1]], #[[2]]} & /@ positions];
Do[output[[position[[1]], position[[2]]]] = 0, {position, positions}];
Do[output[[position[[1]], position[[2]]]] = task["TargetColor"], {position, mapped}]; output
];

BuildQueriesG3[task_Association] := Module[{depth, disabled, specs, trainHashes, rows},
  depth = Length[task["ContextCounts"]];
  disabled = Lookup[task, "DisabledCalibrationLevels", {}];
  specs = Flatten[Table[<|"Kind" -> "CALIBRATE", "Level" -> level, "Prefix" -> prefix|>,
    {level, Select[Range[0, depth - 1], !MemberQ[disabled, #] &]},
    {prefix, AllPrefixesG3[task["ContextCounts"], level]]], 1];
  specs = Join[specs, (<|"Kind" -> "DECISION", "Level" -> Null, "Prefix" -> #|> & /@
    AllPrefixesG3[task["ContextCounts"], depth]),
    If[TrueQ[Lookup[task, "GlobalDecisionProbe", False]],
      <|"Kind" -> "GLOBAL_DECISION", "Level" -> Null, "Prefix" -> {}|>], {}],
    If[TrueQ[Lookup[task, "GlobalBinaryDecisionProbes", False]],
      <|"Kind" -> "GLOBAL_BIT0", "Level" -> Null, "Prefix" -> {}|>,
      <|"Kind" -> "GLOBAL_BIT1", "Level" -> Null, "Prefix" -> {}|>], {}],
    <|"Kind" -> "NUISANCE", "Level" -> Null, "Prefix" -> {}|>];
  trainHashes = GridDigestG3 /@ Lookup[task["InitialTrain"], "Input"];
  rows = Map[Function[spec, With[{grid = MakeInputG3[task, spec]}, <|
    "Kind" -> spec["Kind"], "Level" -> spec["Level"], "Prefix" -> spec["Prefix"],
    "Input" -> grid, "InputSHA256" -> GridDigestG3[grid]|>]], specs];
  SortBy[Select[rows, !MemberQ[trainHashes, #1["InputSHA256"]] &], #1["InputSHA256"] &]
];

TrainingExactG3[task_Association, program_Association] := And @@ Map[
  ApplyProgramG3[task, program, #1["Spec"]] === #1["Output"] &, task["InitialTrain"]];

BuildModelsG3[task_Association, queries_List] := Module[{keyRows, programs, exact},
  keyRows = AllPrefixesG3[task["ContextCounts"], Length[task["ContextCounts"]]];
  programs = Flatten[Table[<|"Keys" -> keys, "Decision" -> decision, "Nuisance" -> nuisance|>,
    {keys, keyRows}, {decision, 0, 2}, {nuisance, 0, task["NuisanceCount"] - 1}], 2];
  exact = Select[programs, TrainingExactG3[task, #1] &];
  Map[Function[program, With[{testSpec = <|"Kind" -> "TEST", "Level" -> Null, "Prefix" -> {}|>},
    With[{prediction = ApplyProgramG3[task, program, testSpec]}, <|
      "ModelKey" -> StringRiffle[ToString /@ Join[program["Keys"], {program["Decision"], program["Nuisance"]}]], ":"],
      "Program" -> program, "DecisionLabel" -> GridDigestG3[prediction], "TestPrediction" -> prediction,
      "QueryPredictions" -> Association@Map[Function[query, query["InputSHA256"] ->
        ApplyProgramG3[task, program, query]], queries]|>]], exact]
];

DecisionCountG3[models_List] := Length[DeleteDuplicates[Lookup[models, "DecisionLabel"]]];
PredictionG3[model_Association, queryHash_String] := model["QueryPredictions"][queryHash];
BranchesG3[models_List, queryHash_String] := GatherBy[models, PredictionG3[#, queryHash] &];
UnsolvedG3[] := <|"Solvable" -> False, "RequiredDepth" -> Null, "FirstInputSHA256" -> Null|>;
$PlanMemoG3 = <||>;
$PlanCountersG4 = <||>;
$QueryByHashG4 = <||>;
$UseConceptsG4 = False;
conceptLibrary = <|"Concepts" -> {}|>;

ResetPlanCountersG4[] := ($PlanCountersG4 = <|
  "ExpandedStateCount" -> 0, "QueryEvaluationCount" -> 0,
  "OutcomeBranchEvaluationCount" -> 0, "ConceptApplicableStateCount" -> 0,
  "ConceptInstantiatedQueryCount" -> 0, "ConceptNoCandidateStateCount" -> 0,
  "ConceptPreferredQueryRejectedCount" -> 0|>);
IncrementCounterG4[key_String, amount_Integer : 1] :=
  ($PlanCountersG4[key] = $PlanCountersG4[key] + amount);

CommonPrefixG4[models_List] := Module[{keys, prefix = {}, values},
  keys = Lookup[Lookup[models, "Program"], "Keys"];
  If[Length[keys] === 0, Return[prefix]];
  Do[values = DeleteDuplicates[keys[[All, index]]];
    If[Length[values] != 1, Break[]]; AppendTo[prefix, First[values]],
    {index, 1, Length[First[keys]]}; prefix
];

RelationalFeaturesS140[models_List, queryHash_String, planningDepth_Integer] := Module[
  {known, coordinateCount, branchRows, branchKnown, branchDecisions, currentDecisions,
  worstKnown, worstDecisions},

```

```

known = Length[CommonPrefixG4[models]];
coordinateCount = Length[First[models][["Program"]]["Keys"]];
branchRows = BranchesG3[models, queryHash];
branchKnown = Length[CommonPrefixG4[#]] & /@ branchRows;
branchDecisions = DecisionCountG3 /@ branchRows;
currentDecisions = DecisionCountG3[models];
worstKnown = If[Length[branchKnown] > 0, Min[branchKnown], known];
worstDecisions = If[Length[branchDecisions] > 0, Max[branchDecisions], currentDecisions];
<|"KnownPrefixLength" -> known, "CoordinateCount" -> coordinateCount,
"RemainingCoordinateCount" -> coordinateCount - known,
"PlanningBudget" -> planningDepth, "CurrentDecisionClassCount" -> currentDecisions,
"WorstCaseKnownPrefixGain" -> worstKnown - known,
"WorstCaseDecisionClassReduction" -> currentDecisions - worstDecisions,
"WorstCaseRemainingDecisionClasses" -> worstDecisions,
"OutcomeBranchCount" -> Length[branchRows]|>
];

AtomMatchesS140[atom_List, features_Association] := Module[{value = features[atom[[1]]],
Switch[atom[[2]], "Equal", value == atom[[3]], "GreaterEqual", value >= atom[[3]],
"LessEqual", value <= atom[[3]], _, False]
];

RuleMatchesS140[rule_Association, features_Association] := And @@
(TrueQ[AtomMatchesS140[#, features]] & /@ rule["Conditions"]);

InventAtomsS140[featureRows_List] := Module[
{atoms = {}, stateFeatures, queryFeatures, operators},
stateFeatures = {"KnownPrefixLength", "CoordinateCount", "RemainingCoordinateCount",
"PlanningBudget", "CurrentDecisionClassCount"};
queryFeatures = {"WorstCaseKnownPrefixGain", "WorstCaseDecisionClassReduction",
"WorstCaseRemainingDecisionClasses", "OutcomeBranchCount"};
operators = {"Equal", "GreaterEqual", "LessEqual"};
Do[AppendTo[atoms, {feature, operator, threshold}],
{feature, Join[stateFeatures, queryFeatures]},
{threshold, Sort[DeleteDuplicates[Lookup[featureRows, feature]]]}, {operator, operators}];
atoms
];

CandidateRulesS140[atoms_List] := Module[{rows, queryFeatures, queryAtoms},
queryFeatures = {"WorstCaseKnownPrefixGain", "WorstCaseDecisionClassReduction",
"WorstCaseRemainingDecisionClasses", "OutcomeBranchCount"};
queryAtoms = Select[atoms, MemberQ[queryFeatures, #[[1]]] &];
rows = (<|"Conditions" -> {#}> & /@ queryAtoms);
Do[If[pair[[1, 1]] != pair[[2, 1]] &&
(MemberQ[queryFeatures, pair[[1, 1]]] || MemberQ[queryFeatures, pair[[2, 1]]]),
AppendTo[rows, <|"Conditions" -> Sort[pair|>]], {pair, Subsets[atoms, {2}]}];
DeleteDuplicatesBy[rows, ToString[#1["Conditions"], InputForm] &]
];

InduceConceptsS139[events_List] := Module[
{positives = {}, negatives = {}, candidates = {}, covered, taskSupport, falseCount,
valid, uncovered, selected = {}, ranked, chosen, body, conceptID, eventIndex = 0,
positiveIndex = 0, featureRows, inventedAtoms},
Do[eventIndex++;
Do[positiveIndex++; AppendTo[positives, <|"EventIndex" -> positiveIndex, "TaskID" -> event["TaskID"],
"Features" -> RelationalFeaturesS140[event["Models"], queryHash, event["PlanningDepth"]]|>],
{queryHash, event["OptimalInputSHA256"]}]];
Do[If[!MemberQ[event["OptimalInputSHA256"], queryHash], AppendTo[negatives,
<|"EventIndex" -> eventIndex, "Features" -> RelationalFeaturesS140[event["Models"], queryHash,
event["PlanningDepth"]]|>]],
{queryHash, event["UnusedInputSHA256"]}], {event, events}];
featureRows = Join[Lookup[positives, "Features"], Lookup[negatives, "Features"]];
inventedAtoms = InventAtomsS140[featureRows];
Do[covered = Select[positives, RuleMatchesS140[rule, #["Features"]] &];
taskSupport = Length[DeleteDuplicates[Lookup[covered, "TaskID"]]];
falseCount = Count[negatives, row_ /; RuleMatchesS140[rule, row["Features"]]];
If[Length[covered] >= protocol["ConceptMinimumSupportEvents"] &&
taskSupport >= protocol["ConceptMinimumDistinctSourceTasks"] && falseCount == 0,
AppendTo[candidates, Join[rule, <|"CoveredEventIndices" -> Lookup[covered, "EventIndex"],
"SupportEventCount" -> Length[covered], "DistinctSourceTaskSupportCount" -> taskSupport,
"TrainingFalsePositiveCount" -> 0|>]], {rule, CandidateRulesS140[inventedAtoms]}];
uncovered = Range[Length[positives]]; valid = candidates;
While[Length[uncovered] > 0,

```

```

ranked = Select[valid, Length[Intersection[#["CoveredEventIndices"], uncovered]] > 0 &];
If[Length[ranked] === 0, Break[]];
ranked = SortBy[ranked, { -Length[Intersection[#["CoveredEventIndices"], uncovered]],
  Length[#["Conditions"]], ToString[#["Conditions"], InputForm]} &];
chosen = First[ranked]; body = <|"Conditions" -> chosen["Conditions"]|>;
conceptID = "KC_" <> StringTake[IntegerString[Hash[chosen["Conditions"], "SHA256"], 16, 64], 16];
AppendTo[selected, Join[<|"ConceptID" -> conceptID|>, body,
  KeyTake[chosen, {"SupportEventCount", "DistinctSourceTaskSupportCount", "TrainingFalsePositiveCount"}]]];
uncovered = Complement[uncovered, chosen["CoveredEventIndices"]];
valid = Select[valid, #["Conditions"] != chosen["Conditions"] &];
];
<|"LibraryType" -> "KERNEL_INVENTED_NUMERIC_RELATIONAL_PREDICATE_CONCEPTS",
  "InductionMethod" -> "DATA_DERIVED_THRESHOLD_ATOMS_PLUS_MDL_ZERO_FALSE_POSITIVE_SET_COVER",
  "PrimitiveNumericStateFeatures" -> {"KnownPrefixLength", "CoordinateCount",
    "RemainingCoordinateCount", "PlanningBudget", "CurrentDecisionClassCount"},
  "PrimitiveNumericQueryFeatures" -> {"WorstCaseKnownPrefixGain",
    "WorstCaseDecisionClassReduction", "WorstCaseRemainingDecisionClasses", "OutcomeBranchCount"},
  "PrimitiveComparisonOperators" -> {"Equal", "GreaterEqual", "LessEqual"},
  "PredeclaredNamedBooleanPredicates" -> {},
  "InventedAtomicPredicateCount" -> Length[inventedAtoms],
  "InventedAtomicPredicates" -> inventedAtoms,
  "SourceEventCount" -> Length[events], "SourceOptimalQueryExampleCount" -> Length[positives],
  "UnabstractedOptimalQueryExampleCount" -> Length[uncovered],
  "ConceptCount" -> Length[selected], "Concepts" -> selected,
  "ConceptsMayPruneModels" -> False, "ConceptsMaySuppressFallbackQueries" -> False,
  "ExactDynamicProgrammingFallbackRequired" -> True|>
];

InstantiateConceptsS139[models_List, unused_List, planningDepth_Integer] := Module[{pairs = {}, matches},
  Do[matches = Select[Sort[unused], RuleMatchesS140[concept,
    RelationalFeaturesS140[models, #, planningDepth]] &];
  Do[If[!MemberQ[Lookup[pairs, "InputSHA256", {}], queryHash], AppendTo[pairs,
    <|"ConceptID" -> concept["ConceptID"], "InputSHA256" -> queryHash|>]], {queryHash, matches}],
  {concept, conceptLibrary["Concepts"]}], pairs
];

PreferredQueriesG4[models_List, unused_List, planningDepth_Integer] := Module[{pairs},
  If[!TrueQ[$UseConceptsG4], Return[{}]];
  pairs = InstantiateConceptsS139[models, unused, planningDepth];
  If[Length[pairs] > 0, IncrementCounterG4["ConceptApplicableStateCount"];
    IncrementCounterG4["ConceptInstantiatedQueryCount", Length[pairs]],
    IncrementCounterG4["ConceptNoCandidateStateCount"]];
  Lookup[pairs, "InputSHA256", {}]
];

SolveAtMostG4[models_List, unused_List, depth_Integer] := Module[
  {key, ordered, preferred, branchRows, remaining, allSolvable, child, result},
  If[DecisionCountG3[models] === 1, Return[<|"Solvable" -> True, "FirstInputSHA256" -> Null|>]];
  If[depth === 0 || Length[unused] === 0, Return[UnsolvedG3[]]];
  key = IntegerString[Hash[{Sort[Lookup[models, "ModelKey"]], Sort[unused], depth}, "SHA256"], 16, 64];
  If[KeyExistsQ[$PlanMemoG3, key], Return[$PlanMemoG3[key]]];
  IncrementCounterG4["ExpandedStateCount"];
  preferred = PreferredQueriesG4[models, Sort[unused], depth];
  ordered = Join[preferred, Select[Sort[unused], !MemberQ[preferred, #] &]];
  result = Catch[
    Do[
      IncrementCounterG4["QueryEvaluationCount"];
      branchRows = BranchesG3[models, queryHash];
      IncrementCounterG4["OutcomeBranchEvaluationCount", Length[branchRows]];
      remaining = DeleteCases[unused, queryHash]; allSolvable = True;
      Do[child = SolveAtMostG4[branch, remaining, depth - 1];
        If[!TrueQ[child["Solvable"]], allSolvable = False; Break[], {branch, branchRows}];
        If[TrueQ[allSolvable],
          Throw[<|"Solvable" -> True, "FirstInputSHA256" -> queryHash|>, "PlanFound"]];
        If[MemberQ[preferred, queryHash], IncrementCounterG4["ConceptPreferredQueryRejectedCount"],
          {queryHash, ordered}];
        UnsolvedG3[],
        "PlanFound"
      ];
    AssociateTo[$PlanMemoG3, key -> result]; result
  ];

FindMinimalPlanG4[models_List, unused_List, maximumDepth_Integer, queryByHash_Association,

```

```

    useConcepts_] := Module[{found, plan},
$PlanMemoG3 = <|>; ResetPlanCountersG4[]; $QueryByHashG4 = queryByHash;
$UseConceptsG4 = TrueQ[useConcepts]; found = UnsolvedG3[];
Do[plan = SolveAtMostG4[models, unused, depth]; If[TrueQ[plan["Solvable"]],
    found = Join[plan, <|"RequiredDepth" -> depth|>]; Break[], {depth, 0, maximumDepth}];
Join[found, <|"WorkCounters" -> $PlanCountersG4|>]
];

OptimalInputsS139[models_List, unused_List, depth_Integer, queryByHash_Association] := Module[
{optimal = {}, remaining, branchRows, allSolvable, child},
$UseConceptsG4 = False; $QueryByHashG4 = queryByHash;
Do[remaining = DeleteCases[unused, queryHash]; branchRows = BranchesG3[models, queryHash];
    allSolvable = True;
    Do[child = SolveAtMostG4[branch, remaining, depth - 1];
        If[!TrueQ[child["Solvable"]], allSolvable = False; Break[], {branch, branchRows}];
        If[TrueQ[allSolvable], AppendTo[optimal, queryHash], {queryHash, Sort[unused]}];
    optimal
];

RunOracleG3[mode_String, taskID_String : "NONE", queryNumber_String : "NONE"] := Module[{command},
    command = "set TCCT_ORACLE_MODE=" <> mode <> "&set TCCT_TASK=" <> taskID <>
        "&set TCCT_QUERY=" <> queryNumber <> "&&" <> protocol["PowerShellEngine"] <>
        " -NoProfile -NonInteractive -EncodedCommand " <> protocol["OracleBridgeEncodedCommand"];
    Run[command]
];

CallOracleG3[taskID_String, queryNumber_String, query_Association] := Module[{request, response, exitCode},
    request = <|"ProtocolSHA256" -> protocolHash, "TaskID" -> taskID, "QueryNumber" -> queryNumber,
        "Input" -> query["Input"], "InputSHA256" -> query["InputSHA256"],
        "GeneratedByTCCTKernel" -> True, "TestOutputAccessed" -> False,
        "HiddenProgramAccessedByLearner" -> False|>;
    Export[oracleRequestPath, request, "RawJSON", "Compact" -> False];
    exitCode = RunOracleG3["query", taskID, queryNumber];
    If[exitCode != 0, Print["FATAL: oracle query failed"]; Exit[20]];
    response = Import[oracleResponsePath, "RawJSON"];
    If[response["ProtocolSHA256"] != protocolHash || response["TaskID"] != taskID ||
        response["QueryNumber"] != queryNumber || response["Input"] != query["Input"] ||
        response["InputSHA256"] != query["InputSHA256"], Print["FATAL: invalid oracle response"]; Exit[21]];
    response
];

$LearningEventsS139 = {};
LearnSourceTaskS139[task_Association] := Module[
{queries, queryByHash, models, initialModels, unused, trace = {}, plan, remainingCap,
    query, queryNumber, response, keep, committed, initialDepth},
    queries = BuildQueriesG3[task]; queryByHash = AssociationThread[Lookup[queries, "InputSHA256"] -> queries];
    models = BuildModelsG3[task, queries]; initialModels = models; unused = Lookup[queries, "InputSHA256"];
    plan = FindMinimalPlanG4[models, unused, protocol["MaximumPlanningDepth"], queryByHash, False];
    initialDepth = If[TrueQ[plan["Solvable"]], plan["RequiredDepth"], Null];
    While[DecisionCountG3[models] > 1 && Length[trace] < protocol["MaximumActiveQueriesPerTask"],
        remainingCap = protocol["MaximumActiveQueriesPerTask"] - Length[trace];
        If[Length[trace] > 0, plan = FindMinimalPlanG4[models, unused, remainingCap, queryByHash, False]];
        If[!TrueQ[plan["Solvable"]] || plan["RequiredDepth"] == 0, Break[]];
        query = queryByHash[plan["FirstInputSHA256"]];
        AppendTo[$LearningEventsS139, <|"TaskID" -> task["TaskID"], "Models" -> models,
            "UnusedInputSHA256" -> unused,
            "PlanningDepth" -> plan["RequiredDepth"],
            "OptimalInputSHA256" -> OptimalInputsS139[models, unused, plan["RequiredDepth"], queryByHash]|>];
        queryNumber = "KQ" <> IntegerString[Length[trace] + 1, 10, 2];
        response = CallOracleG3[task["TaskID"], queryNumber, query];
        keep = PredictionG3[#, query["InputSHA256"]] == response["Output"] & /@ models;
        AppendTo[trace, <|"QueryNumber" -> queryNumber, "InputSHA256" -> query["InputSHA256"],
            "QueryKind" -> query["Kind"], "QueryLevel" -> query["Level"], "QueryPrefix" -> query["Prefix"],
            "DecisionClassCountBefore" -> DecisionCountG3[models],
            "DecisionClassCountAfter" -> DecisionCountG3[Pick[models, keep, True]],
            "GeneratedByTCCTKernel" -> True|>];
        models = Pick[models, keep, True]; unused = DeleteCases[unused, query["InputSHA256"]];
    ];
    committed = DecisionCountG3[models] == 1;
    <|"TaskID" -> task["TaskID"], "InitialSemanticClassCount" -> Length[initialModels],
        "InitialCertifiedMinimumDepth" -> initialDepth, "ActiveQueryCount" -> Length[trace],
        "ActiveQueryTrace" -> trace, "DecisionCertified" -> committed,
        "ConceptLabelsAccessed" -> False, "ConceptBodiesAccessed" -> False|>
];

```

```
];
```

```
EvaluateTaskG3[task_Association] := Module[
{queries, queryByHash, models, initialModels, unused, trace = {}, initialGuided, initialBaseline,
guided, baseline, remainingCap, query, response, keep, queryNumber, stopReason, committed,
runtime, value, rootPairs, guidedTotal = 0, baselineTotal = 0,
guidedStates = 0, baselineStates = 0, parity = True},
queries = BuildQueriesG3[task]; queryByHash = AssociationThread[Lookup[queries, "InputSHA256"] -> queries];
models = BuildModelsG3[task, queries]; initialModels = models; unused = Lookup[queries, "InputSHA256"];
initialGuided = FindMinimalPlanG4[models, unused, protocol["MaximumPlanningDepth"], queryByHash, True];
initialBaseline = FindMinimalPlanG4[models, unused, protocol["MaximumPlanningDepth"], queryByHash, False];
parity = initialGuided["Solvable"] === initialBaseline["Solvable"] &&
initialGuided["RequiredDepth"] === initialBaseline["RequiredDepth"];
guidedTotal += initialGuided["WorkCounters"]["QueryEvaluationCount"];
baselineTotal += initialBaseline["WorkCounters"]["QueryEvaluationCount"];
guidedStates += initialGuided["WorkCounters"]["ExpandedStateCount"];
baselineStates += initialBaseline["WorkCounters"]["ExpandedStateCount"];
{runtime, value} = AbsoluteTiming[While[DecisionCountG3[models] > 1 && Length[trace] < protocol["MaximumActiveQueriesPerTask"],
remainingCap = Min[protocol["MaximumPlanningDepth"], protocol["MaximumActiveQueriesPerTask"] - Length[trace]];
If[Length[trace] === 0, guided = initialGuided; baseline = initialBaseline,
guided = FindMinimalPlanG4[models, unused, remainingCap, queryByHash, True];
baseline = FindMinimalPlanG4[models, unused, remainingCap, queryByHash, False];
parity = parity && guided["Solvable"] === baseline["Solvable"] &&
guided["RequiredDepth"] === baseline["RequiredDepth"];
guidedTotal += guided["WorkCounters"]["QueryEvaluationCount"];
baselineTotal += baseline["WorkCounters"]["QueryEvaluationCount"];
guidedStates += guided["WorkCounters"]["ExpandedStateCount"];
baselineStates += baseline["WorkCounters"]["ExpandedStateCount"];
If[!TrueQ[guided["Solvable"]] || guided["RequiredDepth"] === 0,
stopReason = "NO_PLAN_WITHIN_RESOURCE_DEPTH"; Break[]];
rootPairs = InstantiateConceptsS139[models, unused, guided["RequiredDepth"]];
query = queryByHash[guided["FirstInputSHA256"]];
queryNumber = "KQ" <> IntegerString[Length[trace] + 1, 10, 2];
response = CallOracleG3[task["TaskID"], queryNumber, query];
keep = PredictionG3[#1, query["InputSHA256"]] === response["Output"] & /@ models;
AppendTo[trace, <|"QueryNumber" -> queryNumber, "Input" -> query["Input"],
"InputSHA256" -> query["InputSHA256"], "QueryKind" -> query["Kind"],
"QueryLevel" -> query["Level"], "QueryPrefix" -> query["Prefix"],
"AdmissionMode" -> "CONCEPT_GUIDED_EXACT_DP_WITH_FALLBACK",
"CertifiedMinimumDepthBefore" -> guided["RequiredDepth"],
"BaselineCertifiedMinimumDepthBefore" -> baseline["RequiredDepth"],
"BaselineFirstInputSHA256" -> baseline["FirstInputSHA256"],
"RootConceptMatched" -> (Length[rootPairs] > 0),
"RootConceptIDs" -> DeleteDuplicates[Lookup[rootPairs, "ConceptID", {}]],
"RootConceptPreferredInputSHA256" -> Lookup[rootPairs, "InputSHA256", {}],
"SelectedQueryWasRootConceptPreference" -> MemberQ[Lookup[rootPairs, "InputSHA256", {}], query["InputSHA256"]],
"RootConceptFallbackUsed" -> (Length[rootPairs] === 0),
"GuidedWorkCounters" -> guided["WorkCounters"],
"BaselineWorkCounters" -> baseline["WorkCounters"],
"DecisionClassCountBefore" -> DecisionCountG3[models], "SemanticClassCountBefore" -> Length[models],
"OracleOutput" -> response["Output"], "DecisionClassCountAfter" -> DecisionCountG3[Pick[models, keep, True]],
"SemanticClassCountAfter" -> Length[Pick[models, keep, True]], "GeneratedByTCCTKernel" -> True,
"TestOutputAccessed" -> False, "HiddenProgramAccessedByLearner" -> False];
models = Pick[models, keep, True]; unused = DeleteCases[unused, query["InputSHA256"]];
}];
committed = DecisionCountG3[models] === 1;
If[!StringQ[stopReason], stopReason = If[committed, "DECISION_CERTIFIED", "NO_PLAN_WITHIN_RESOURCE_DEPTH"];
<|"TaskID" -> task["TaskID"], "RuntimeSeconds" -> runtime,
"InitialSemanticClassCount" -> Length[initialModels], "InitialDecisionClassCount" -> DecisionCountG3[initialModels],
"InitialPlanExistsWithinCap" -> TrueQ[initialGuided["Solvable"]],
"InitialCertifiedMinimumDepth" -> If[TrueQ[initialGuided["Solvable"]], initialGuided["RequiredDepth"], Null],
"InitialBaselineCertifiedMinimumDepth" -> If[TrueQ[initialBaseline["Solvable"]], initialBaseline["RequiredDepth"], Null],
"PairedDepthParity" -> parity,
"InitialGuidedWorkCounters" -> initialGuided["WorkCounters"],
"InitialBaselineWorkCounters" -> initialBaseline["WorkCounters"],
"GuidedQueryEvaluationCount" -> guidedTotal,
"BaselineQueryEvaluationCount" -> baselineTotal,
"GuidedExpandedStateCount" -> guidedStates,
"BaselineExpandedStateCount" -> baselineStates,
"FinalSemanticClassCount" -> Length[models], "FinalDecisionClassCount" -> DecisionCountG3[models],
"ActiveQueryCount" -> Length[trace], "ActiveQueryTrace" -> trace, "AdaptiveStopReason" -> stopReason,
"DecisionCertified" -> committed, "TestPredictionCommitted" -> committed,
"CommittedTestPrediction" -> If[committed, First[models]["TestPrediction"], Null],
```

```

    "Status" -> If[committed, "DECISION_CERTIFIED", "NO_PLAN_WITHIN_RESOURCE_DEPTH"],
    "TestOutputAccessed" -> False, "HiddenProgramAccessedByLearner" -> False]>
];

Print[protocol["Stage"]];
Print["Native kernel forms anonymous relational planning concepts before opening target tasks"];
resetCode = RunOracleG3["reset"];
If[resetCode != 0, Print["FATAL: oracle reset failed"]; Exit[22]];
{sourceRuntime, sourceRows} = AbsoluteTiming[LearnSourceTaskS139 /@ public["SourceTasks"]];
sourceDepths = Lookup[sourceRows, "InitialCertifiedMinimumDepth"];
sourceQueryCount = Total[Lookup[sourceRows, "ActiveQueryCount"]];
conceptLibrary = InduceConceptsS139[$LearningEventsS139];
conceptLibrary = Join[conceptLibrary, <|"CreatedByNativeWolframKernel" -> True,
    "CreatedAfterSourceQueryCount" -> sourceQueryCount, "CreatedBeforeTargetTaskExecution" -> True,
    "GeneratorProvidedConceptLabelCount" -> 0, "GeneratorProvidedConceptBodyCount" -> 0|>];
Export[conceptLibraryPath, conceptLibrary, "RawJSON", "Compact" -> False];
conceptLibraryHash = IntegerString[FileHash[conceptLibraryPath, "SHA256"], 16, 64];
Print["source depths=", sourceDepths, " events=", Length[$LearningEventsS139],
    " concepts=", conceptLibrary["ConceptCount"]];
{targetRuntime, taskRows} = AbsoluteTiming[EvaluateTaskG3 /@ public["TargetTasks"]];
Do[Print[Row["TaskID", " depth=", row["InitialCertifiedMinimumDepth"], " queries=",
    row["ActiveQueryCount"], " status=", row["Status"]], {row, taskRows}];
initialDepths = Lookup[taskRows, "InitialCertifiedMinimumDepth"];
tracePolicyPass = And @@ Flatten[Map[Function[row, Map[Function[q,
    q["AdmissionMode"] == "CONCEPT_GUIDED_EXACT_DP_WITH_FALLBACK" &&
    IntegerQ[q["CertifiedMinimumDepthBefore"]] && q["CertifiedMinimumDepthBefore"] >= 1 &&
    q["BaselineCertifiedMinimumDepthBefore"] == q["CertifiedMinimumDepthBefore"]],
    row["ActiveQueryTrace"]], taskRows]];
commitCount = Count[Lookup[taskRows, "TestPredictionCommitted"], True];
oracleLogCount = Length@Select[StringSplit[Import[oracleLogPath, "Text"], "\n"], StringLength[#] > 0 &];
guidedQueryEvaluations = Total[Lookup[taskRows, "GuidedQueryEvaluationCount"]];
baselineQueryEvaluations = Total[Lookup[taskRows, "BaselineQueryEvaluationCount"]];
pairedDepthParity = And @@ Lookup[taskRows, "PairedDepthParity"];
rootConceptUseCount = Count[Flatten[Lookup[taskRows, "ActiveQueryTrace"], 1], q_ /;
    TrueQ[q["SelectedQueryWasRootConceptPreference"]]];
rootFallbackCount = Count[Flatten[Lookup[taskRows, "ActiveQueryTrace"], 1], q_ /;
    TrueQ[q["RootConceptFallbackUsed"]]];
preferredRejectionCount = Total[Lookup[Lookup[taskRows, "InitialGuidedWorkCounters"],
    "ConceptPreferredQueryRejectedCount"];
conceptSupportPass = conceptLibrary["ConceptCount"] >= 1 && And @@
    ({["SupportEventCount"] >= 2 && #["DistinctSourceTaskSupportCount"] >= 2 &&
    #["TrainingFalsePositiveCount"] == 0 & /@ conceptLibrary["Concepts"]];
preScorePass = sourceDepths == {3, 2, 1} && sourceQueryCount >= 3 &&
    Length[$LearningEventsS139] >= 3 && conceptSupportPass &&
    Sort[initialDepths] == {1, 2, 2, 2, 3} && tracePolicyPass && commitCount == 5 &&
    pairedDepthParity && guidedQueryEvaluations < baselineQueryEvaluations &&
    rootConceptUseCount >= 1 && rootFallbackCount >= 1 &&
    oracleLogCount == sourceQueryCount + Total[Lookup[taskRows, "ActiveQueryCount"]] &&
    !MemberQ[Lookup[taskRows, "TestOutputAccessed"], True] &&
    !MemberQ[Lookup[taskRows, "HiddenProgramAccessedByLearner"], True];
result = <|"Stage" -> protocol["Stage"], "EvidenceStatus" -> protocol["EvidenceStatus"],
    "ProtocolSHA256" -> protocolHash, "WolframRunnerSHA256" -> runnerHash,
    "OracleRespondersSHA256" -> oracleHash, "KernelInducedConceptLibrarySHA256" -> conceptLibraryHash,
    "NativeWolframExecution" -> True, "WolframVersion" -> $Version,
    "RuntimeSeconds" -> sourceRuntime + targetRuntime, "SourceTaskResults" -> sourceRows,
    "SourcePlanningEventCount" -> Length[$LearningEventsS139],
    "KernelInducedConceptLibrary" -> conceptLibrary, "TargetTaskResults" -> taskRows,
    "ObservedTargetActiveQueryCounts" -> Lookup[taskRows, "ActiveQueryCount"],
    "ObservedTargetCertifiedDepths" -> initialDepths,
    "AggregateGuidedQueryEvaluationCount" -> guidedQueryEvaluations,
    "AggregateBaselineQueryEvaluationCount" -> baselineQueryEvaluations,
    "DeterministicPlanningWorkReduced" -> (guidedQueryEvaluations < baselineQueryEvaluations),
    "PairedDepthParity" -> pairedDepthParity, "RootConceptSelectedQueryCount" -> rootConceptUseCount,
    "RootConceptFallbackCount" -> rootFallbackCount,
    "ConceptPreferredQueryRejectedCount" -> preferredRejectionCount,
    "OracleQueryLogLineCount" -> oracleLogCount, "TracePolicyPreScorePass" -> tracePolicyPass,
    "ConceptBodiesCreatedByNativeKernel" -> True, "ConceptBodiesProvidedByGenerator" -> False,
    "NativePreScorePass" -> preScorePass, "CoreRewriteFreezeDedupModified" -> False,
    "Conclusion" -> If[preScorePass,
        "KERNEL_NATIVE_CONCEPT_FORMATION_COMPLETE_AWAITING_SEALED_SCORE", "NATIVE_PREScore_FAILURE"]]>;
Export[resultPath, result, "RawJSON", "Compact" -> False];
Print["native pre-score pass=", preScorePass, " runtime=", sourceRuntime + targetRuntime];
Exit[0];

```


Primary aggregate execution result

```
{
  "Stage": "S140-B DIAGNOSTIC R0 cross-family aggregate",
  "ReplicationMasterSHA256": "67df83fb87a7b564e1fa5053b99880c270079b92b76ac95b5290255fe333c178",
  "CohortPassCount": 2,
  "FreshTargetWorldCount": 15,
  "ExactTargetPassCount": 15,
  "DistinctSelectedConceptBodySetCount": 3,
  "TotalGuidedWork": 10569,
  "TotalBaselineWork": 14616,
  "PlanningWorkReductionFraction": 0.27688834154351394,
  "Cohorts": [
    {
      "CohortID": "C01",
      "GeneratorFamily": "MIXED_PREFIX",
      "Seed": 1402101,
      "InventedAtomicPredicateCount": 60,
      "SelectedConceptCount": 2,
      "UnabstractedOptimalQueryExampleCount": 0,
      "SelectedConcepts": [
        {
          "ConceptID": "KC_9367b954b4f705d3",
          "Conditions": [
            [
              "KnownPrefixLength",
              "GreaterEqual",
              2
            ],
            [
              "OutcomeBranchCount",
              "Equal",
              3
            ]
          ],
          "SupportEventCount": 6,
          "DistinctSourceTaskSupportCount": 3,
          "TrainingFalsePositiveCount": 0
        },
        {
          "ConceptID": "KC_ed4b21f1f97412bc",
          "Conditions": [
            [
              "WorstCaseKnownPrefixGain",
              "Equal",
              1
            ]
          ],
          "SupportEventCount": 3,
          "DistinctSourceTaskSupportCount": 2,
          "TrainingFalsePositiveCount": 0
        }
      ],
      "ExactTargetPassCount": 5,
      "GuidedWork": 996,
      "BaselineWork": 9296,
      "AdversarialRejected": true,
      "AdversarialSafelyNotActivated": false,
      "Checks": {
        "FrozenIdentityPass": true,
        "ReceiptAndMaterializationBoundaryPass": true,
        "AtLeastOneZeroFalsePositiveConcept": true,
        "FiveTargetsExact": true,
        "AllRoleDepthChecksPass": true,
        "TransferPlanningWorkReduced": true,
        "AdversarialControlSafe": true,
        "FallbackExercised": true,
        "NativeIndependentConceptMatch": true,
        "PredicateVerificationPass": true,
        "DifficultyAuditPass": true,

```

```

    "CoreUnchanged": true
  },
  "CohortPass": true
},
{
  "CohortID": "C02",
  "GeneratorFamily": "HIGH_PREFIX",
  "Seed": 1402137,
  "InventedAtomicPredicateCount": 63,
  "SelectedConceptCount": 2,
  "UnabstractedOptimalQueryExampleCount": 0,
  "SelectedConcepts": [
    {
      "ConceptID": "KC_f5985bcf11108178",
      "Conditions": [
        [
          "OutcomeBranchCount",
          "Equal",
          3
        ]
      ],
      "SupportEventCount": 10,
      "DistinctSourceTaskSupportCount": 3,
      "TrainingFalsePositiveCount": 0
    },
    {
      "ConceptID": "KC_ed4b21f1f97412bc",
      "Conditions": [
        [
          "WorstCaseKnownPrefixGain",
          "Equal",
          1
        ]
      ],
      "SupportEventCount": 4,
      "DistinctSourceTaskSupportCount": 2,
      "TrainingFalsePositiveCount": 0
    }
  ],
  "ExactTargetPassCount": 5,
  "GuidedWork": 9390,
  "BaselineWork": 4482,
  "AdversarialRejected": true,
  "AdversarialSafelyNotActivated": false,
  "Checks": {
    "FrozenIdentityPass": true,
    "ReceiptAndMaterializationBoundaryPass": true,
    "AtLeastOneZeroFalsePositiveConcept": true,
    "FiveTargetsExact": true,
    "AllRoleDepthChecksPass": true,
    "TransferPlanningWorkReduced": false,
    "AdversarialControlSafe": true,
    "FallbackExercised": false,
    "NativeIndependentConceptMatch": true,
    "PredicateVerificationPass": false,
    "DifficultyAuditPass": false,
    "CoreUnchanged": true
  },
  "CohortPass": false
},
{
  "CohortID": "C03",
  "GeneratorFamily": "LOW_PREFIX",
  "Seed": 1402179,
  "InventedAtomicPredicateCount": 66,
  "SelectedConceptCount": 2,
  "UnabstractedOptimalQueryExampleCount": 2,
  "SelectedConcepts": [
    {
      "ConceptID": "KC_e6b8db56d023855b",
      "Conditions": [
        [
          "PlanningBudget",

```

```

        "GreaterEqual",
        2
    ],
    [
        "WorstCaseKnownPrefixGain",
        "Equal",
        1
    ]
],
"SupportEventCount": 5,
"DistinctSourceTaskSupportCount": 2,
"TrainingFalsePositiveCount": 0
},
{
    "ConceptID": "KC_2a26fd498d5e8db0",
    "Conditions": [
        [
            "WorstCaseDecisionClassReduction",
            "Equal",
            2
        ]
    ],
    "SupportEventCount": 3,
    "DistinctSourceTaskSupportCount": 3,
    "TrainingFalsePositiveCount": 0
}
],
"ExactTargetPassCount": 5,
"GuidedWork": 183,
"BaselineWork": 838,
"AdversarialRejected": true,
"AdversarialSafelyNotActivated": false,
"Checks": {
    "FrozenIdentityPass": true,
    "ReceiptAndMaterializationBoundaryPass": true,
    "AtLeastOneZeroFalsePositiveConcept": true,
    "FiveTargetsExact": true,
    "AllRoleDepthChecksPass": true,
    "TransferPlanningWorkReduced": true,
    "AdversarialControlSafe": true,
    "FallbackExercised": true,
    "NativeIndependentConceptMatch": true,
    "PredicateVerificationPass": true,
    "DifficultyAuditPass": true,
    "CoreUnchanged": true
},
"CohortPass": true
}
],
"Checks": {
    "AllThreeFamiliesPassWithoutReplacement": false,
    "AllFifteenFreshTargetsExact": true,
    "AtLeastTwoDistinctConceptBodySets": true,
    "EveryFamilyHasAtLeastOneConcept": true,
    "EveryFamilyTransferWorkReduced": false,
    "NoFilteringOrReplacement": true,
    "CoreUnchanged": true
},
"AggregateGatePass": false,
"CoreRewriteFreezeDedupModified": false,
"Conclusion": "CROSS_FAMILY_BOUNDED_PREDICATE_INVENTION_FAIL"
}

```

Independent cross-family verification

```
{
  "Stage": "S140-B DIAGNOSTIC R0 independent cross-family verification",
  "ReplicationMasterSHA256": "67df83fb87a7b564e1fa5053b99880c270079b92b76ac95b5290255fe333c178",
  "CohortIntegrity": [
    {
      "CohortID": "C01",
      "Checks": {
        "MasterLinkPass": true,
        "IdentityPass": true,
        "CopiedLockedHashesPass": true,
        "BaseVerificationPass": true,
        "PredicateVerificationPass": true
      },
      "Pass": true
    },
    {
      "CohortID": "C02",
      "Checks": {
        "MasterLinkPass": true,
        "IdentityPass": true,
        "CopiedLockedHashesPass": true,
        "BaseVerificationPass": false,
        "PredicateVerificationPass": false
      },
      "Pass": false
    },
    {
      "CohortID": "C03",
      "Checks": {
        "MasterLinkPass": true,
        "IdentityPass": true,
        "CopiedLockedHashesPass": true,
        "BaseVerificationPass": true,
        "PredicateVerificationPass": true
      },
      "Pass": true
    }
  ],
  "Checks": {
    "MasterFrozenBeforeMaterialization": true,
    "LockedSourceHashesPass": true,
    "OrchestrationHashesPass": true,
    "AllCohortIntegrityChecksPass": false,
    "AggregateReplayPass": true,
    "AggregateGatePass": false,
    "CoreUnchanged": true
  },
  "IndependentVerificationPass": false,
  "CoreRewriteFreezeDedupModified": false,
  "OpenEndedLanguageInventionClaimed": false,
  "Conclusion": "INDEPENDENT_CROSS_FAMILY_VERIFICATION_FAIL"
}
```